Library Management System Software Specification

Borrower profiles, book inventory and borrowing

Version 1.0 | 29 October 2026 | Draft for stakeholder review

Purpose and intended outcome

Build a web application for a library. A Librarian manages borrower profiles and the book inventory. Borrowers sign in to view their own profile, borrow books and track their loans. Anyone, including visitors who are not signed in, can browse the All Books page. Whenever a borrower borrows a book, the inventory updates automatically. This document defines observable behavior and verification criteria for an initial release, suitable for planning, AI-assisted implementation, and testing.

Source requirement

> *"The Librarian can access every borrower's information, while a borrower can only view his own profile. Only the Librarian can create, edit or remove a borrower's profile. An inventory holds information about all books, and only the Librarian can add a new book. The inventory updates automatically whenever a borrower borrows a book."*

Source: the user-provided brief for a Library Management System. The brief also identifies React for the frontend, FastAPI for the API, PostgreSQL for the database, the e-mail domains @lib.sample.com (Librarian) and @bor.sample.com (Borrower), a publicly accessible All Books page, and a redirect to sign in when a visitor tries to borrow a book.

How to read this specification

Requirements marked U come directly from the user brief. Requirements marked P are proposed elaborations needed to make the brief implementable and testable. P requirements are a coherent draft baseline, not stakeholder-approved facts. All field lengths, loan terms, limits, error behavior, performance targets, and operational policies are proposed unless explicitly identified as U.

"Shall" describes required behavior within this draft baseline. Examples illustrate the contract; they do not add features. Requirement IDs connect implementation work, acceptance criteria, and tests. The product need does not require an AI chatbot or AI-powered recommendations; AI-assisted development is only a possible delivery approach.

Document map

Section	Coverage
1–2	Scope, assumptions, actors and access
3–5	Functional requirements, borrowing workflow and screens
6–8	Data model, REST interfaces and reliability
9–10	Quality requirements, acceptance criteria and test cases
11	Delivery sequence, change control and decisions

1 Scope and baseline assumptions

Included in the initial release

Librarians create, edit, remove and read borrower profiles, add books to the inventory, view inventory figures, and record returned books. Borrowers read their own profile, borrow books and read their own loans. Visitors browse All Books. The application keeps a persistent loan history and an append-only inventory event log. All user-facing actions are enforced by server-side permissions.

ID	Basis	Requirement
U01	U	The system shall be a web application with a React frontend, a FastAPI REST API and a PostgreSQL database.
U02	U	The Librarian shall be able to access the information of every borrower.
U03	U	A Borrower shall be able to view only his or her own profile.
U04	U	Only the Librarian shall be able to create, edit or remove a borrower’s profile.
U05	U	The system shall include an inventory component holding information about all books.
U06	U	Only the Librarian shall be able to add a new book to the inventory.
U07	U	The inventory shall be updated automatically whenever a Borrower borrows a book.
U08	U	Librarian accounts shall use e-mail addresses in the domain @lib.sample.com; Borrower accounts shall use @bor.sample.com.
U09	U	An All Books page shall be accessible to anyone, including users who are not signed in.
U10	U	When a user who is not signed in tries to borrow a book, the user shall be redirected to sign in.

Proposed assumptions

A01 — This release serves one library. Each account has exactly one application role: Librarian or Borrower. Librarian accounts are provisioned outside the application (database seed or deployment script); there is no Librarian management screen. Borrower accounts are created only by a Librarian. There is no self-registration.

A02 — The inventory tracks each book title with a total number of copies and a number of available copies. Individual physical copies, barcodes and shelf locations are not tracked.

A03 — Borrower account status is Active or Removed. Loan status is Borrowed or Returned. Overdue is a derived indicator (Borrowed and past the due date), not a stored status. The loan period is 14 days and a Borrower may hold at most 5 active loans. These values are proposed.

A04 — A Librarian records the return of a book. Without a return action the inventory could only shrink, so this proposed action completes the borrowing cycle. Borrowers cannot return books themselves. Librarians cannot borrow books.

A05 — Authentication uses e-mail and password managed by the application, because Librarians create borrower accounts. Connecting an external identity provider is a stakeholder decision (Section 11).

Excluded from this release

Borrower self-registration, borrower self-editing of profiles, editing or removing books, reservations and holds, renewals, fines and payments, e-mail/SMS/push notifications, individual copy and barcode tracking, book cover uploads, e-books, multiple branches, reports and dashboards, Librarian management screens, hard deletion of any record, and AI recommendations are excluded. The application shall not show menu items for excluded features.

2 Actors and access control

P01 — Every page and API operation other than the public operations shall require an authenticated, active account. The public operations are: `GET /books`, `GET /books/{id}` and `POST /auth/login`. The server derives the user ID and role from validated identity context. Client-supplied identity or role values shall never determine access.

Capability	Visitor	Borrower	Librarian
Browse All Books and book details	Yes	Yes	Yes
Search and filter books; see available copy count	Yes	Yes	Yes
See total copies, copies on loan and inventory events	No	No	Yes
Add a new book to the inventory	No	No	Yes
Borrow a book	No (redirected to sign in)	Yes, for self	No
Create a borrower profile	No	No	Yes
List and search borrower profiles	No	No	Yes, all
Read a borrower profile	No	Own profile only	Any profile
Edit a borrower profile	No	No	Any Active profile
Remove a borrower profile	No	No	Any Active profile, no active loans
List and read loans	No	Own loans only	All loans
Record the return of a book	No	No	Yes
Change own password	No	Yes	Yes
Edit or delete a book; delete a loan or history	No	No	No

Authorization rules

P02 — The role of an account is determined by its e-mail domain: the part after the last “@”, compared case-insensitively. @lib.sample.com identifies a Librarian and @bor.sample.com identifies a Borrower. The server enforces this when an account is created (422 for any other domain) and again at every sign-in; a mismatch between stored role and domain is rejected with the same generic 401 as a wrong password. A role is immutable after creation.

P03 — Ownership of a profile or loan is the immutable user ID or borrower_id stored at creation. A Borrower shall see only his or her own profile and loans, including through search, pagination totals and filters. A Librarian shall see all. UI visibility is a convenience; the API must independently enforce the same rules.

P04 — Unauthenticated or expired sessions shall receive HTTP 401. Authenticated users attempting a role-restricted endpoint shall receive 403. Requests for a nonexistent resource or one the user cannot read shall return the same 404 response. Authorization is checked before mutation. Removed users cannot authenticate; an existing session of a removed user is rejected on its next request. Historical references to a removed user remain visible to Librarians.

P05 — A borrow attempt without authentication returns 401 from the API. The UI shall redirect the visitor to Sign in, remember the book page as the return target, and after successful sign-in return the user to that book page. The borrow itself shall never run automatically; a signed-in Borrower must press Borrow again. If the account that signs in is a Librarian, the book page opens without a Borrow action.

Primary user journeys

Visitor: open All Books → search or filter → open a book → press Borrow → redirected to Sign in → sign in with a @bor.sample.com account → return to the book → press Borrow → receive loan and due date → see availability decrease.

Borrower: sign in → open My Profile to read own details → open My Loans → see due dates, overdue indicators and returned books.

Librarian: sign in → open Borrowers → create, edit or remove a profile → open Inventory → add a new book → open All Loans → record a returned book → confirm the inventory count increased.

3 Functional requirements

Borrower profiles

FR01 [U02, U04] — On valid submission by a Librarian, the server shall create one Borrower account with a unique immutable UUID, a readable member number, a role of Borrower, status Active, the supplied profile details, a temporary password stored only as a hash, must_change_password set to true, and server-generated UTC timestamps. A success response shall contain the saved profile; the form shall navigate to its details.

FR02 [P] — Name, e-mail, optional phone and address, and the temporary password shall pass the field rules in Section 6. Missing, blank, out-of-range, wrong-domain, duplicate or unknown fields shall fail validation without creating an account. No partial record shall remain after a failed transaction.

FR03 [U02] — Borrowers shall list all borrower profiles with search by name, e-mail or member number, a status filter and pagination. The Librarian profile detail shall show all profile fields and the borrower’s current and past loans.

FR04 [U03] — My Profile shall display the signed-in Borrower’s own member number, name, e-mail, phone, address and status as read-only information. A Borrower shall have no way to read another profile or edit any profile field.

FR05 [U04; P rules] — A Librarian shall edit name, e-mail, phone, address, or reset the temporary password of an Active profile using a partial update. At least one supported field shall be supplied. Every modification shall use the profile version to prevent overwriting a newer change. A password reset sets must_change_password to true.

FR06 [U04; P rules] — A Librarian shall remove an Active profile only when the borrower has no active loans. Removal is a soft removal: status becomes Removed, removed_at is set, sign-in is blocked, and loan history is preserved (rule P10).

Books, inventory and the All Books page

FR07 [U09] — All Books shall be reachable without signing in. It shall list every book with title, author, category and available copies, and support search by title, author or ISBN, a category filter, an available-only filter, and pagination. A book with no available copies shall be shown as Not available with the Borrow action disabled.

FR08 [U09] — Book Details shall be reachable without signing in and shall show ISBN, title, author, publisher, category, publication year and available copies. It shall never show total copies, loan counts or borrower information to visitors or Borrowers.

FR09 [U05] — Inventory shall be available only to Librarians. It shall list all books with ISBN, title, author, category, total copies, available copies, copies on loan and added date, with search and filters for category and out-of-stock books, and a per-book history of inventory events.

FR10 [U06] — A Librarian shall add a new book by supplying ISBN, title, author, optional publisher, category, publication year and total copies. The server shall set available copies equal to total copies, record an Added inventory event, and reject duplicate ISBNs. No other role may add books.

Borrowing and loans

FR11 [U07, U10] — An authenticated Active Borrower shall borrow an available book. On success the server creates a loan with status Borrowed and a due date, decreases the book’s available copies by one and records a Borrowed inventory event, all in one transaction (rules P06–P09). Visitors are redirected to sign in (rule P05).

FR12 [P] — My Loans shall display book title, borrowed date, due date, returned date and status, with an Overdue indicator, and support a status filter. All Loans (Librarian) shall add borrower name and member number plus borrower and book filters.

FR13 [P] — A Librarian shall record the return of a Borrowed loan. The server sets status Returned and returned_at, increases the book’s available copies by one and records a Returned inventory event atomically.

FR14 [U07; P] — Every change to a book’s available copies shall be accompanied by exactly one immutable inventory event with actor, quantity change, resulting available count and server time. Inventory events are append-only.

Authentication

FR15 [U08; P] — Users shall sign in with e-mail and password and sign out. After sign-in with must_change_password true, the only permitted actions are changing the password and signing out; all other operations return 403 PASSWORD_CHANGE_REQUIRED. Five consecutive failed sign-ins for one e-mail lock sign-in for that e-mail for 15 minutes.

4 Borrowing lifecycle and business rules

From	To	Conditions
New borrow	Borrowed	Server-controlled. Active Borrower, book has an available copy, fewer than 5 active loans, no active loan for the same book. Due date set by server.
Borrowed	Returned	Librarian only. Sets returned_at and returns one copy to the inventory.
Returned	(any)	Terminal. A returned loan is never reopened; borrowing again creates a new loan.

P06 — A borrow request shall be rejected, with no change to data, when: the caller is a Librarian (403); the book does not exist (404); no copy is available (409 OUT_OF_STOCK); the Borrower already has 5 active loans (409 BORROW_LIMIT_REACHED); or the Borrower already has an active loan for the same book (409 ALREADY_BORROWED).

P07 — The due date is the server date in UTC when the loan is created plus 14 days. It cannot be supplied or changed by the client. A loan is Overdue when its status is Borrowed and the current UTC date is later than its due date. Overdue loans do not block further borrowing in this release.

P08 — The loan, the decrease in available copies, the Borrowed inventory event and the idempotency record shall commit atomically. The server shall lock the book row (or use an equivalent conditional update) so that available copies never fall below zero. If two Borrowers request the last copy at the same time, exactly one succeeds and the other receives 409 OUT_OF_STOCK. If any required write fails, all writes roll back. A successful response must reflect the committed state.

P09 — Recording a return shall be Librarian-only. The loan update, the increase in available copies, the Returned event and the idempotency data shall commit atomically. Available copies shall never exceed total copies. Returning an already Returned loan returns 409 INVALID_TRANSITION and changes nothing.

P10 — Removing a borrower is allowed only for an Active profile with zero loans in status Borrowed; otherwise the API returns 409 HAS_ACTIVE_LOANS. Removal sets status Removed and removed_at, keeps the e-mail address reserved, preserves all loans, and makes the profile read-only: any edit of a Removed profile returns 409 PROFILE_REMOVED. Restoring a removed profile is not included.

P11 — An e-mail change must keep the @bor.sample.com domain and remain unique ignoring case. Role, member number, status and password hash cannot be supplied in a profile update; unknown fields are rejected. A patch containing only unchanged values returns the current profile without incrementing its version.

P12 — An ISBN is unique after removing hyphens and spaces. A duplicate ISBN returns 409 DUPLICATE_ISBN. A duplicate e-mail on profile creation or edit returns 409 DUPLICATE_EMAIL.

Example scenario

The Librarian adds “Clean Architecture” with 3 copies. All Books shows 3 available. A visitor, Anita, opens the book and presses Borrow. She is redirected to Sign in, signs in as anita@bor.sample.com, returns to the book page and presses Borrow. A loan is created with a due date 14 days ahead and All Books now shows 2 available. When Anita returns the book, the Librarian records the return and availability becomes 3 again. Both changes appear in the book’s inventory history.

Handling repeat submissions

P13 — The UI disables submit actions while a request is pending. The API additionally supports an Idempotency-Key for creating borrowers, books and loans. Retrying the same request with the same key shall return the original result without creating duplicate records or changing inventory twice. Detailed rules appear in Section 8.

5 Web interface specification

Screen	Required content and behavior
All Books	Public. Book list with title, author, category, available copies; search, category and available-only filters, pagination. Borrow action per book. Default sort is title ascending.
Book Details	Public. ISBN, title, author, publisher, category, year, availability. Borrow action. Visitors are redirected to Sign in.
Sign in	E-mail and password. Generic error for any failure. Returns to the intended page after authentication. Forced password change screen when required.
My Profile	Borrower only. Own details, read-only. No edit controls.
My Loans	Borrower only. Own loans, status filter, due dates and Overdue indicator.
Borrowers	Librarian only. All profiles, search, status filter, Create Borrower action, link to details.
Borrower Form	Librarian only. Create and edit with labelled fields, field-level errors, and temporary password on create or reset.
Borrower Details	Librarian only. Full profile, version, loans, Edit and Remove actions (Remove needs a confirmation dialog).
Inventory	Librarian only. All books with total, available and on-loan counts; Add Book form; per-book event history.
All Loans	Librarian only. All loans with filters and a Record Return action with confirmation.

Form behavior

UI01 — Required fields shall be visibly labelled. Whitespace-only input shall fail validation. Client-side validation shall mirror server-side rules, but the server remains authoritative. Text entries shall be retained when validation or network errors occur.

UI02 — A successful borrow shall show the due date and the updated availability. The page shall never imply success until the server confirms the write. On timeout, retain the original idempotency key so Retry can safely recover the result.

UI03 — On a version conflict while editing or removing a profile, display “This profile has changed. Refresh it before saving.” Preserve unsent input separately, load current data, and require the user to submit a revised action. Never silently overwrite another change.

Empty, loading and error states

UI04 — Empty lists shall say “No books found”, “No borrowers found” or “No loans found” and allow users to clear filters. Loading indicators shall appear while data is loading. Errors shall offer a retry action without exposing stack traces or database details. When a borrow fails with OUT_OF_STOCK, ALREADY_BORROWED or BORROW_LIMIT_REACHED, the page shall show a plain-English message.

UI05 — A 401 response shall return the user to Sign in. A 403 shall explain that the action is unavailable. A 404 shall show “Not found or unavailable.” Failed mutations shall leave displayed persisted values unchanged until a fresh response is available.

Navigation and usability

UI06 — Visitor navigation shall include All Books and Sign in. Borrower navigation shall include All Books, My Loans and My Profile. Librarian navigation shall include All Books, Borrowers, Inventory and All Loans. Route guards hide unauthorized screens as a convenience only. All forms and controls shall support keyboard use, visible focus, associated labels and readable validation messages. Availability and status must be expressed as text as well as color.

UI07 — Dates are stored in UTC and displayed in the user’s browser timezone with an accessible timezone indicator. All Books, Inventory, My Loans and All Loans shall refresh automatically every 30 seconds while active, and support manual refresh. Background tabs may pause polling and refresh when activated.

6 Data model and validation

All identifiers below are opaque UUIDs unless stated otherwise. Text length means Unicode code points after trimming leading and trailing whitespace. Text is stored as plain text and escaped on display. Timestamps use ISO 8601 UTC values.

User (account and borrower profile)

Field	Type and rule
id / member_number	Server UUID / unique number MB- plus at least six decimal digits for Borrowers; null for Librarians. Numbers may have gaps and shall never be reused.
full_name	Required string, 2–100 characters.
email	Required, 5–254 characters, valid address, stored lowercase, unique ignoring case. Domain must be @lib.sample.com (Librarian) or @bor.sample.com (Borrower).
role	Enum Librarian, Borrower; immutable; must match the e-mail domain.
phone	Optional; 7–15 digits with an optional leading +.
address	Optional string, up to 300 characters.
status	Enum Active, Removed. Server defaults to Active.
password_hash	Salted slow hash (argon2id or bcrypt). Never returned or logged. Password rule: 12–128 characters and different from the e-mail.
must_change_password	Boolean; true after creation or reset by a Librarian.
failed_login_count / locked_until	Server-managed sign-in lockout state.
created_at / updated_at / removed_at	Server timestamps. created_at immutable; removed_at set on removal.
version	Positive integer; starts at 1 and increments on committed profile changes.

Book (inventory record)

Field	Type and rule
id / book_code	Server UUID / unique code BK- plus at least six decimal digits.
isbn	Required; 10 or 13 digits after removing hyphens and spaces; unique.
title	Required string, 1–200 characters.
author	Required string, 1–150 characters.
publisher	Optional string, up to 150 characters.
category	Required string, 2–60 characters.
publication_year	Required integer from 1000 to the current year.
total_copies	Required integer 1–1000 on creation; not editable in this release.
available_copies	Integer 0 to total_copies. Server-managed; equals total_copies on creation; changed only by borrow and return.
created_by / created_at / updated_at	Librarian user key and server timestamps.

Related records

Loan: id, loan_number (LN- plus at least six digits, unique), book_id, borrower_id, borrowed_at, due_date, returned_at (nullable), returned_by (nullable Librarian), status (Borrowed or Returned). Loans cannot be edited except by the return action and cannot be deleted.

InventoryEvent: id, book_id, event_type (Added, Borrowed or Returned), quantity_delta, available_after, loan_id (nullable), actor_id, occurred_at. Events are append-only. Added events have a positive delta equal to total copies; Borrowed has −1; Returned has +1.

IdempotencyRecord: user_id, operation, key, request_hash, response_status, response_body and expires_at. Enforce uniqueness on user_id plus operation plus key for its retention window.

Persistence constraints

Enforce unique IDs, e-mail, member number, ISBN, loan number, valid enum values, required fields and foreign keys at persistence boundaries. Add database CHECK constraints so that 0 ≤ available_copies ≤ total_copies, and a partial unique index on (borrower_id, book_id) where status is Borrowed. Index email, role, status, isbn, title, author, category, loans.borrower_id, loans.book_id, loans.status and loans.due_date. Loans and events shall not be orphaned; no deletion endpoint is included for any record.

7 REST API contract

Base path: /api/v1. Requests and responses use application/json. Endpoints require authentication unless marked Public. Lists return {items, page, page_size, total}. Default page=1 and page_size=20; maximum page_size=100. Invalid parameters return 422. Account responses never contain password data.

Method and path	Contract
POST /auth/login	Public. Body: email, password. 200 sets an HttpOnly session cookie and returns id, email, role, full_name, must_change_password. 401 INVALID_CREDENTIALS on any failure; 429 when locked out.
POST /auth/logout	Authenticated. 204; session ends.
POST /auth/change-password	Authenticated. Body: current_password, new_password. 204; clears must_change_password.
GET /me	Authenticated. Returns the current account.
GET /books	Public. Optional q, category, available=true and pagination. Returns public book fields only (no total_copies). 200.
GET /books/{id}	Public. Public book fields or 404.
POST /books	Librarian-only. Body: isbn, title, author, publisher, category, publication_year, total_copies. Idempotency-Key required. Returns 201, book and Location header.
GET /inventory	Librarian-only. Full book fields including total_copies, available_copies, on_loan. Optional q, category, out_of_stock=true and pagination.
GET /inventory/{book_id}/events	Librarian-only. Inventory events, oldest first by occurred_at then id; paginated.
POST /borrowers	Librarian-only. Body: full_name, email, phone, address, temporary_password. Idempotency-Key required. Returns 201, profile, ETag and Location.
GET /borrowers	Librarian-only. Optional q, status=active|removed and pagination. Returns 200.
GET /borrowers/{id}	Librarian: any profile. Borrower: own profile only, any other ID returns 404. Includes ETag holding its quoted version.
PATCH /borrowers/{id}	Librarian-only. Body subset: full_name, email, phone, address, temporary_password. If-Match required. Returns 200 with saved profile and ETag.
DELETE /borrowers/{id}	Librarian-only. If-Match required. Soft removal per rule P10. Returns 200 with the Removed profile.
POST /loans	Borrower-only. Body: book_id. Idempotency-Key required. Returns 201 with the loan and the book's new available_copies. Unauthenticated callers receive 401.
GET /loans	Borrower: own loans. Librarian: all loans with optional borrower_id, book_id, status=borrowed|returned|overdue. Paginated.
GET /loans/{id}	Own loan for a Borrower, any loan for a Librarian; otherwise 404.
POST /loans/{id}/return	Librarian-only. No body. Returns 200 with the Returned loan and updated available_copies.

Filtering and ordering

API01 — q is a literal, case-insensitive substring match, limited to 100 trimmed characters. For books it matches title, author or ISBN; for borrowers it matches name, e-mail or member number. Empty q means no text filter. Multiple filters combine with AND. Books order by title ascending then id; borrowers by name then id; loans by borrowed_at descending then id descending. total counts only matching authorized records. Pagination uses the current state; records may move between pages after concurrent updates.

Request examples

Add book: `{"isbn":"978-0-13-449416-6","title":"Clean Architecture","author":"Robert C. Martin","publisher":"Pearson","category":"Software","publication_year":2017,"total_copies":3}`

Create borrower: `{"full_name":"Anita Rao","email":"anita@bor.sample.com","phone":"+919800000000","temporary_password":"Welcome-2026-xyz"}`

Borrow: `POST /loans {"book_id":"<book uuid>"}` with an Idempotency-Key header.

8 Errors, inventory integrity and reliability

Error contract

All errors use `{"error":{"code":"...","message":"...","fields":{},"request_id":"..."}}`. fields maps invalid field names to plain-English messages; it is empty for nonvalidation errors. Unknown JSON fields are rejected, including role, status or available_copies in a create request.

HTTP	Code and meaning
400	MALFORMED_JSON — request cannot be parsed.
401	UNAUTHENTICATED / INVALID_CREDENTIALS — missing or expired identity, or failed sign-in (generic message).
403	FORBIDDEN / PASSWORD_CHANGE_REQUIRED — disallowed role or action; password change pending.
404	NOT_FOUND — nonexistent or inaccessible resource.
409	OUT_OF_STOCK, BORROW_LIMIT_REACHED, ALREADY_BORROWED, HAS_ACTIVE_LOANS, PROFILE_REMOVED, DUPLICATE_EMAIL, DUPLICATE_ISBN, INVALID_TRANSITION or IDEMPOTENCY_CONFLICT — business rule conflict, or key reused with a different request.
412 / 428	VERSION_CONFLICT / PRECONDITION_REQUIRED — stale If-Match or required If-Match missing.
422	VALIDATION_ERROR — invalid fields, filters, enum, domain or missing idempotency key.
429	TOO_MANY_REQUESTS — sign-in lockout or public rate limit exceeded.
500 / 503	INTERNAL_ERROR / UNAVAILABLE — unexpected failure or temporary dependency outage; no internal details.

Concurrent updates and retries

API02 — For profile updates and removals, If-Match shall contain the quoted integer version returned by profile reads. Version comparison and update happen atomically. A stale version returns 412 with no mutation. A malformed version returns 422. Authorization is checked before version details are exposed. Borrowing and returning rely on row locking instead of If-Match.

API03 — Idempotency-Key is a client-generated UUID, scoped by user and operation, retained for 24 hours. An exact replay returns the original status and body; a different body returns 409. Simultaneous requests with the same key are serialized. Replays recheck current authentication and authorization. Validation failures do not consume keys; after expiry, a new submission can create a new record.

Inventory integrity rules

INV01 — For every book, available_copies must equal total_copies minus the number of its loans in status Borrowed. Only three operations may change available_copies: adding a book (initial value), borrowing (−1) and recording a return (+1). Each writes exactly one InventoryEvent. A reconciliation query comparing counts, loans and events shall return no differences after any test sequence.

INV02 — Updated availability shall be available through the API immediately after commit and shall appear on an active UI within the 30-second refresh interval under normal service conditions.

INV03 — Failure to write the inventory event shall roll back the associated mutation under this single-database design. No external system is notified in this release.

9 Quality and operational requirements

The following are proposed release targets. They are measurable acceptance gates, not claims about an existing system. Infrastructure sizing and production ownership must be settled before deployment.

ID	Requirement and verification
NF01 Security	Use HTTPS outside local development. Validate identity, authorization and input on every request; use parameterized database access. Escape rendered text. Test script-like payloads remain inert. Rate-limit public endpoints per IP (proposed 120 requests per minute).
NF02 Sessions	Hash passwords with argon2id or bcrypt; never store or log plain passwords. Use short-lived signed session tokens in Secure, HttpOnly cookies with appropriate SameSite settings and CSRF protection for writes. Restrict allowed web origins (CORS).
NF03 Performance	In a stated test environment with 10,000 books, 2,000 borrowers, 50,000 loans and 50 concurrent users, p95 read responses ≤ 1 second and writes ≤ 2 seconds over 15 minutes; unexpected failures < 1%.
NF04 Usability	At viewport widths 375 and 1440 pixels, all primary actions remain usable without page-level horizontal scrolling. Test keyboard-only browsing, borrowing, profile creation and adding a book.
NF05 Durability	A successful write survives application restart. Automated daily PostgreSQL backups; proposed recovery point ≤ 24 hours and recovery time ≤ 4 hours. Verify a restore before production launch.
NF06 Observability	Log request ID, route, status, duration and operational error category. Do not log passwords, tokens or personal profile details. Monitor API failure rate and database availability.
NF07 Consistency	Loan, inventory update, inventory event and idempotency success record commit in a single transaction. Simulated write failure shall leave no partial result.
NF08 Verification	Automate API validation, permission, lifecycle, retry and concurrency tests. Run UI smoke tests for Visitor, Borrower and Librarian flows before release.

Architecture boundaries

A React web frontend calls the FastAPI REST API. The API owns authorization, validation, workflow and transactions. A PostgreSQL database persists users, books, loans, inventory events and idempotency records. Unlike general specifications, the three technologies are fixed by U01; supporting libraries (for example SQLAlchemy, Alembic, Pydantic, a React router and a test framework) remain implementation decisions and shall not weaken this contract. No external e-mail service is required for the proposed initial release.

Deployment and data handling

Maintain separate development, test and production configuration; keep secrets outside source control. Apply versioned database migrations with a tested backup and rollback procedure. The runtime account shall have only necessary database permissions. Borrower names, e-mail addresses, phone numbers and addresses are personal data. Define retention and administrative data-removal procedures before production; soft-removed profiles keep their data until such a procedure exists, and the absence of a hard-delete button does not establish indefinite retention.

10 Acceptance criteria and test cases

Tests use a Librarian account, separate Borrower A and Borrower B accounts, a removed Borrower account, and an unauthenticated visitor. Seed books with distinct availability, including one with a single copy and one with zero available. Each test below is mandatory for this proposed baseline; verify persisted state as well as the HTTP response and visible UI behavior.

Test	Given and action	Expected result and trace
TC01	Visitor opens All Books and a Book Details page.	200 without sign-in; only public fields; no total_copies. U09, FR07–FR08.
TC02	Visitor searches, filters and pages All Books; send page_size 101.	Correct matches, order and total; 422 for invalid parameter. FR07, API01.
TC03	Visitor presses Borrow; call POST /loans without sign-in.	UI redirects to Sign in with return target; API 401; no loan; inventory unchanged. U10, P05.
TC04	After redirect, Borrower signs in.	Returns to the book page; no loan exists until Borrow is pressed again. P05.
TC05	Borrower borrows an available book.	201; Borrowed; due date +14 days; available −1; one Borrowed event. U07, P06–P08.
TC06	Borrow a book with zero available copies.	409 OUT_OF_STOCK; no loan; no event. P06.
TC07	Two Borrowers borrow the last copy at once.	One 201, one 409; available is 0, never negative. P08.
TC08	Borrow the same book twice; borrow a sixth active loan.	409 ALREADY_BORROWED; 409 BORROW_LIMIT_REACHED; state unchanged. P06.
TC09	Librarian attempts to borrow.	403; no loan. P06.
TC10	Librarian creates a borrower with valid data.	201; role Borrower; member number; Active; must_change_password true. U04, FR01.
TC11	Create with blank name, @lib.sample.com or @gmail.com e-mail, malformed or duplicate e-mail, short password, or unknown field role.	422 or 409 with field error; zero new records. FR02, P02, P12.
TC12	Test name lengths 1, 2, 100, 101; password lengths 11, 12, 128, 129.	Only inclusive allowed boundaries pass. FR02, Section 6.
TC13	Borrower attempts to create, edit or remove any profile, including own.	403; state unchanged. U04.
TC14	Borrower A reads own profile, then B’s profile, then lists borrowers.	200; 404; 403. U03, P03.
TC15	Librarian lists, searches and opens any borrower with loans.	All profiles visible; correct totals and loan history. U02, FR03.
TC16	Librarian edits with current version; stale version; missing If-Match; unchanged values.	200 and version +1; 412; 428; 200 with no version change. FR05, P11, API02.
TC17	Edit e-mail to another domain or to an existing e-mail.	422; 409 DUPLICATE_EMAIL; profile unchanged. P02, P11.
TC18	Remove a borrower with an active loan; then without.	409 HAS_ACTIVE_LOANS; then 200 Removed, sign-in blocked, session rejected, history kept. FR06, P04, P10.
TC19	Edit a Removed profile.	409 PROFILE_REMOVED. P10.
TC20	Sign in as Librarian and Borrower; wrong password; account whose role mismatches its domain; five failures.	Valid sign-ins succeed; 401 generic; 401 generic; then 429 lockout. P02, FR15.
TC21	First sign-in with a temporary password.	403 PASSWORD_CHANGE_REQUIRED on all but change-password until changed. FR15.
TC22	Librarian adds a valid book.	201; available = total; Added event; visible on All Books and Inventory. U06, FR10.
TC23	Add book with missing fields, bad ISBN, duplicate ISBN, unknown field or total 0.	422 or 409 DUPLICATE_ISBN; zero records. FR10, P12.
TC24	Test title 0, 1, 200, 201 characters; total_copies 0, 1, 1000, 1001; future year.	Only inclusive allowed boundaries pass. Section 6.

Test	Given and action	Expected result and trace
TC25	Borrower and visitor attempt POST /books and GET /inventory.	403 for Borrower; 401 for visitor; no data. U06, P01, P04.
TC26	Librarian opens Inventory and a book’s events.	Totals, on-loan counts and event history are correct. FR09, FR14.
TC27	Librarian records a return; repeat; Borrower tries.	200, available +1, Returned event; 409 INVALID_TRANSITION; 403. FR13, P09.
TC28	Run mixed and concurrent borrows and returns; run reconciliation.	available = total − active loans for every book. INV01.
TC29	Force database failure when inserting the inventory event during a borrow.	Entire mutation rolls back; retry can complete once. P08, INV03, NF07.
TC30	Replay a create or borrow with identical key and body; reuse key with different body.	Identical replay returns original result; changed body 409; no duplicates. P13, API03.
TC31	Borrower reads loans; Borrower A requests B’s loan; Librarian lists all.	Own loans only; 404; all loans with filters. P03, FR12.
TC32	Enter HTML/script-like content in book title, borrower name and address.	Stored as text within limits; rendered inert. NF01.
TC33	Use keyboard and narrow viewport for all three journeys.	Labels, focus, errors and all actions usable. UI06, NF04.
TC34	Run the 15-minute load and restore a backup in test.	Meet p95/error targets; restore within targets with recorded evidence. NF03, NF05.
TC35	Borrow in one session; watch All Books in another.	Availability updates within the refresh target. UI07, INV02.

Overall release acceptance

All U requirements and adopted P requirements shall be traced to passing tests. The visitor-to-loan journey, the borrower journey and the librarian journey shall pass end to end. No unresolved defect may permit unauthorized access, a Borrower reading another profile, a non-Librarian changing a profile or the inventory, a negative or inconsistent available count, lost committed data, or duplicate writes within the retry window. Record test environment, build identifier, execution date and evidence. This document specifies tests; it does not claim that any implementation has passed them.

11 Delivery plan and decisions

Implementation sequence

Task 1 — Establish the PostgreSQL schema and migrations, authentication, the e-mail domain and role rules, and isolated test users. Verify access boundaries before exposing profile data.

Task 2 — Implement borrower profile creation, editing with version checks, soft removal, scoped reads, and the Borrowers, My Profile and Borrower Form screens. Verify field rules and ownership.

Task 3 — Implement books, the inventory component, the public All Books and Book Details pages, and Add Book. Verify public access and Librarian-only restrictions.

Task 4 — Implement borrowing, the sign-in redirect, returns, inventory events and idempotency. Verify the automatic inventory update, concurrency on the last copy, and reconciliation.

Task 5 — Complete error states, accessibility and operational controls. Execute all acceptance tests and document results. Resolve stakeholder decisions before production rollout.

Decisions requiring stakeholder review

Decision	Proposed baseline	Effect if changed
Identity and provisioning	App-managed e-mail and password; Librarians seeded; Borrowers created by a Librarian.	An external identity provider changes authentication integration and the profile-creation flow.
Loan terms	14-day loan, 5 active loans, no renewals, no fines.	Renewals, fines or holds expand scope and tests.
Return handling	Librarian records returns; Borrowers cannot self-return.	Self-return or scanning needs new permissions and tests.
Book maintenance	Librarian adds books only; no edit, remove or add-copies action.	Editing or stock adjustment needs new rules and inventory events.
Borrower removal	Soft removal, blocked while loans are active, no restore.	Hard deletion or restore needs retention and legal review.
Public catalogue	Visitors see book details and available count only.	Showing total copies or loans publicly changes the privacy boundary.
Update channels	No notifications; availability by refresh.	E-mail reminders need delivery, retry and privacy contracts.
Production operations	Proposed load and recovery targets in Section 9.	Owner must confirm sizing, service hours, retention and recovery duties.

Specification change control

Store this specification with the implementation. For a requirement change, update the requirement, affected API/data contracts and tests before changing code. Record version, date, rationale and affected IDs.
AI-generated code shall be reviewed against this baseline; a tool prompt does not override the specification.

Definition of ready

The brief is elaborated enough for an implementation exercise using the stated assumptions. For a production baseline, the product owner must resolve the decisions above and approve scope. The operational owner must accept hosting, retention and recovery responsibilities. No stakeholder approval is asserted by this draft.